AMANE — Refonte V3

Faire d'AMANE une enveloppe d'exécution souveraine pour SaaS externes

Journal de refonte, décisions d'architecture et tri du dépôt

Version	1.2
Date	2026-10-28
Branche	main3
Statut	Étape 1 terminée — bascule HA (étape 2) à venir

Sommaire

Fiche du document 3

Historique des versions . . . 3

1. Objet de la refonte 4

1.1 Ce qu'est une « enveloppe d'exécution » . . . 4

1.2 La promesse à tenir . . . 4

2. État actuel du dépôt (V2, point de départ) 5

2.1 Point de départ figé . . . 5

2.2 Cartographie des composants . . . 5

2.3 Trois constats qui orientent le tri . . . 5

3. Décisions d'architecture 7

ADR-001 — Haute disponibilité PME : etcd + Patroni piloté par l'orchestrateur Go . . . 7

ADR-002 — Formats d'exécutables acceptés . . . 7

ADR-003 — Zero-knowledge strict . . . 8

4. Tri du dépôt 9

4.1 À garder intact . . . 9

4.2 À garder mais adapter . . . 9

4.3 À supprimer . . . 10

5. Plan de travail par étapes 11

6. Journal des étapes 12

Étape 0 — Préparation (2026-10-28) . . . 12

Étape 1 — Nettoyage (2026-10-28) . . . 12

Étape 1 — Compléments (2026-10-28) . . . 13

7. Points ouverts 14

Annexe — Mettre à jour ce document 15

Fiche du document

Ce document est le **journal de bord de la refonte V3** d'AMANE. Il récapitule l'état du dépôt au départ, le tri de chaque composant (garder intact, adapter, supprimer), les décisions d'architecture prises et leur justification, puis l'avancement étape par étape. Il sert de référence aux collaborateurs pour comprendre **ce qui a été fait, pourquoi, et dans quel ordre**.

Il est mis à jour à chaque étape de la refonte. La source fait foi (`docs/refonte-v3/AMANE-Refonte-V3.md`) ; le PDF est régénéré à partir d'elle (voir l'annexe « Mettre à jour ce document »).

Historique des versions

Version	Date	Branche / commit	Changements
1.0	2026-10-28	main3 / 6da5022	Création : état V2, tri initial, décisions ADR-001 à ADR-003, plan par étapes
1.1	2026-10-28	main3 / dc06197	Étape 1 (nettoyage) terminée ; tri corrigé (§ 4.3) ; points à vérifier ajoutés (§ 7)
1.2	2026-10-28	main3	Compléments à l'étape 1 : compilation Rust confirmée, test de réplication corrigé, test V1 `mock_server.rs` retiré ; réplication synchrone ajoutée à ADR-001 et à l'étape 2

1. Objet de la refonte

1.1 Ce qu'est une « enveloppe d'exécution »

Jusqu'en V2, AMANE embarquait sa propre application métier (gestion de stock) au-dessus de son socle de souveraineté. La V3 inverse la logique : **AMANE devient l'enveloppe dans laquelle s'exécute un SaaS édité par un tiers**.

- Un éditeur externe soumet l'**exécutable** de son SaaS à AMANE (en priorité une image Docker).
- AMANE le déploie sur le cluster de la PME et en assure l'**orchestration** : haute disponibilité, réplication, bascule, réseau, sauvegarde.
- Le SaaS s'exécute **sur la base de l'architecture AMANE** : ses données restent chez la PME et ne sortent que chiffrées.
- L'**interface utilisateur et la couche métier du SaaS ne changent pas**. L'utilisateur final retrouve exactement l'application qu'il connaît ; l'éditeur externe n'a pas à réécrire son code ni à intégrer un SDK.

1.2 La promesse à tenir

> **Personne d'autre que la PME ne voit de données en clair. La DEK et toutes les clés de chiffrement restent dans le cluster PME. Ni l'éditeur AMANE, ni le relais, ni l'éditeur du SaaS externe n'ont accès au clair.**

Cette promesse est le cœur d'AMANE ; toute décision de la refonte est évaluée à son aune.

2. État actuel du dépôt (V2, point de départ)

2.1 Point de départ figé

- Branche de travail créée : `main3`, à partir de `main2` (commit `e0f6d09`), poussée sur `origin/main3`.
- Le travail V2 non commité (76 fichiers) a été figé dans le commit `6da5022` « chore: état V2 avant nettoyage ». C'est le point de retour de référence avant toute suppression.

2.2 Cartographie des composants

Composant	Chemin	Rôle en V2
SaaS éditeur (Django)	`config/`, `tenants/`, `licenses/`, `devices/`, `dashboard/`, `api/`, `templates/`	Comptes tenants, licences, parc installé, génération des installeurs PME
Console web (React)	`frontend/`	Console éditeur et espace tenant, consomme l'API DRF
Orchestrateur cluster (Go)	`orchestrator-go/`	etcd v3, Patroni, réplication CRDT, mesh WireGuard, serveur gRPC mTLS
Socle crypto et journal (Rust)	`spike/crates/ss-crypto`, `ss-journal`, `ss-consensus`	AK/DEK, CRL, Shamir, journal CBOR chiffré, époque et fencing, garde de quorum
SDK client métier (Rust)	`core-rust/ss-client/`	Invariants stock, mode hors-ligne, appairage, licence, client gRPC
Nœud PME (Rust)	`spike/node/`	CLI, supervision PostgreSQL, annonce au relais, **application stock et utilisateurs intégrée**
Relais zero-knowledge (Rust)	`spike/relay/`, `relay-deploy/`	Stockage de blobs opaques et topologie du cluster
Contrat gRPC	`proto/amane/framework/v1/framework.proto`	Interface unique B ↔ C (Ping, Enroll, Write, Read, NotifyRevocation, PushDelta)
Déploiement PME	`pme-deploy/`, `templates/tenants/*.tmpl`	PostgreSQL primaire/standby + nœud + pgAdmin + watchtower
CI et mesures	`.github/workflows/`, `.ci/`, `scripts/`, `tests/`, `docs/bench/`	Build, couverture, chaos, contrats, benchmarks, mesures de bascule

2.3 Trois constats qui orientent le tri

Constat 1 — Deux mécanismes de haute disponibilité coexistent. L'orchestrateur Go (etcd + Patroni) n'est utilisé que dans le `docker-compose.yml` de développement et dans la CI. La pile réellement déployée chez la PME n'en contient pas : elle repose sur une réplication streaming PostgreSQL nue et sur le nœud Rust, qui décide seul de la promotion d'un standby (`spike/crates/ss-consensus/src/quorum.rs`, élection déterministe par LSN). Cette élection écrite à la main est en tension avec la règle n° 2 du projet (« aucun algorithme de consensus écrit à la main »).

Constat 2 — Le nœud PME embarque sa propre application métier. Le module `spike/node/src/stock/` et les pages HTML associées (articles, utilisateurs, tableau de bord) constituent une application métier complète. C'est précisément le rôle que tiendra désormais le SaaS externe.

Constat 3 — La console éditeur existe en double. Une version rendue côté serveur par Django (`templates/*.html`, `views_web.py`, `views.py`) et une version React (`frontend/`) couvrent les mêmes

écrans.

Autres observations : le nœud déclare la dépendance `ss-client` dans le workspace mais ne l'utilise pas ; un fichier proto V1 (`proto/framework.proto`) déclare le même paquet que le contrat qui fait autorité ; plusieurs scripts et notes isolés traînent à la racine.

3. Décisions d'architecture

Chaque décision est consignée sous forme d'ADR (Architecture Decision Record) : contexte, décision, conséquences. Une décision n'est jamais effacée : si elle est remplacée, elle passe au statut « Remplacée par ADR-xxx ».

ADR	Titre	Statut	Date
ADR-001	Haute disponibilité PME : etcd + Patroni piloté par l'orchestrateur Go	Acceptée	2026-10-28
ADR-002	Formats d'exécutables acceptés : Docker d'abord, binaires et .exe ensuite	Acceptée	2026-10-28
ADR-003	Zero-knowledge strict : aucun clair hors du cluster PME, clés confinées	Acceptée	2026-10-28

ADR-001 — Haute disponibilité PME : etcd + Patroni piloté par l'orchestrateur Go

Contexte. Deux mécanismes de HA coexistent (constat 1). La garde de quorum Rust (`quorum.rs`) réalise une élection de primaire écrite à la main, ce que la règle n° 2 interdit. L'orchestrateur Go, lui, s'appuie sur etcd (Raft) et Patroni, est déjà couvert par les tests d'intégration et par les mesures de bascule de la CI (switchover ~2,2 s, failover ~3,2 s).

Décision. La pile PME adopte **etcd + Patroni, piloté par l'orchestrateur Go**. C'est le seul mécanisme de bascule et de fencing en production.

Conséquences.

- Les compose PME (`pme-deploy/`, `templates/tenants/docker-compose.yml.tmpl`) intègrent etcd, Patroni (spilo) et l'orchestrateur Go, à l'image du compose de développement.
- `ss-consensus/src/quorum.rs` et la boucle de supervision/promotion du nœud Rust (`supervision.rs`, `run_supervision_loop`, `cmd/failover.rs`) sont retirés ou réduits à la lecture d'état ; le nœud ne promeut plus PostgreSQL lui-même.
- La règle « 2 nœuds → bascule manuelle ; ≥ 3 nœuds → failover automatique par quorum » est conservée et portée par etcd/Patroni. Avec 2 nœuds etcd n'a pas de quorum tolérant à une panne : la bascule reste manuelle et le SaaS signale à la PME qu'elle est sous le seuil.
- L'époque et le fencing (`epoch.rs`, `fencing.rs`) restent dans le socle : les écritures restent conditionnées à la lease de leader etcd.
- **Réplication synchrone effective.** La pile V2 positionne `synchronous_commit=remote_write` sans définir `synchronous_standby_names` : PostgreSQL valide alors chaque transaction sur la seule écriture locale, la réplication est en réalité asynchrone. Cela contredit la règle « synchrone pour les invariants forts, blocage si aucun passif n'est joignable ». L'étape 2 la rend effective via Patroni (`synchronous_mode` et `synchronous_mode_strict`), qui bloque les écritures plutôt que de dégrader silencieusement en asynchrone (décision du 2026-10-28).

ADR-002 — Formats d'exécutables acceptés

Contexte. Un SaaS externe peut être livré sous plusieurs formes : image Docker, binaire Linux, exécutable Windows (.exe). Supporter tous les formats d'emblée disperserait l'effort.

Décision. AMANE accueillera **plusieurs types d'exécutables**. L'effort porte d'abord **exclusivement sur les images Docker**. Les binaires et les .exe seront développés ensuite.

Conséquences.

- Le format de description d'une application soumise (le « manifeste ») porte dès le départ un champ **type d'artefact** (`docker`, puis `binaire`, `exe`), afin que l'ajout ultérieur des autres formats n'oblige pas à casser le modèle de données ni le contrat.
- Les images sont référencées par **digest** (`sha256:…`) et non par tag, pour que la PME exécute exactement ce qui a été soumis.
- Piste pour la suite (non tranchée) : envelopper un binaire nu dans une image de base fournie par AMANE ; un .exe Windows demandera des conteneurs Windows ou une machine virtuelle.

ADR-003 — Zero-knowledge strict

Contexte. La promesse d'AMANE est que seule la PME voit ses données en clair. En V3 un acteur nouveau entre dans le périmètre : l'éditeur du SaaS externe, dont le code s'exécute **à l'intérieur** du cluster PME et manipule donc du clair.

Décision. Le zero-knowledge est maintenu sans compromis : **personne d'autre que la PME ne voit de clair**, et **la DEK comme toutes les clés de chiffrement restent dans le cluster PME**.

Conséquences.

- **Confinement réseau** : l'application externe s'exécute sur un réseau interne sans sortie vers Internet par défaut. Seuls sont exposés son interface utilisateur (via le reverse proxy du nœud) et sa base de données gérée par AMANE. Elle ne peut pas renvoyer de données en clair à son éditeur.
- **Chiffrement au repos** : les données de l'application (base, volumes) sont chiffrées sous des clés dérivées de la hiérarchie AMANE ; la DEK ne quitte jamais le cluster.
- **Sortie uniquement chiffrée** : sauvegardes, réplication inter-sites et tout transit par le relais sont chiffrés côté PME avant envoi. Le relais ne voit que des blobs opaques.
- **Télémétrie** : le SaaS éditeur AMANE ne reçoit que des métadonnées (état du cluster, licence, santé), jamais de contenu métier.
- **Aucun composant hors PME** (Django, relais, éditeur externe, agent IA) ne touche AK/DEK. Les opérations sur les clés restent humaines.

4. Tri du dépôt

4.1 À garder intact

Composant	Justification
`spike/crates/ss-crypto` (AK, DEK, CRL, Shamir, device_key, recovery)	Socle de la souveraineté, indépendant de tout métier
`spike/crates/ss-journal`	Journal CBOR chiffré append-only, générique
`ss-consensus/src/epoch.rs`, `fencing.rs`	Anti split-brain générique (lease etcd)
`spike/relay/`, `relay-deploy/`	Le relais zero-knowledge convient tel quel
`proto/amane/framework/v1/`, `tests/contracts/`, `buf.gen.yaml`	Contrat figé : on pourra ajouter des messages, jamais modifier l'existant
`orchestrator-go/` (consensus, supervisor, grpcserver, mesh, replication, telemetry)	Devient le moteur HA de production (ADR-001)
Modèles `Tenant`, `License`, `Device` ; `devices/api_register.py` ; `api/`	Comptes, licences et identité d'installation inchangés
`.github/workflows/`, `.ci/`, `scripts/*_measure.sh`, `docs/bench/`	Garde-fous CI et mesures de bascule

4.2 À garder mais adapter

Composant	Adaptation prévue
`spike/node`	Devient l'**hôte d'exécution** : garde init, run, status, delist, le client relais et le heartbeat SaaS ; gagne le lancement et la supervision de l'application externe à partir de son manifeste, la base PostgreSQL gérée, et un reverse proxy qui sert l'interface de l'application telle quelle
`core-rust/ss-client`	N'est plus un SDK intégré par l'application (elle reste inchangée). `license.rs`, `pairing.rs` et `offline.rs` sont rapatriés dans le nœud ou conservés comme bibliothèque interne
Django (`tenants/`, `licenses/`, `devices/`, `dashboard/`)	Nouveau modèle **Application** (image soumise, digest, manifeste : ports, volumes, base requise, variables), rattaché à la licence ; génération des installeurs et du compose incluant l'application
`templates/tenants/*.tmpl`, `pme-deploy/`	Ajout du service applicatif et de la pile etcd/Patroni/orchestrateur (ADR-001) ; `watchtower` revu (mise à jour par digest vérifié uniquement) ; `pgadmin` rendu optionnel
`frontend/`	Garde Tenants, Licences, Appareils, Clusters, Relais, MonEspace, Login, Register, Déploiement ; ajoute une page Applications ; met à jour la page Architecture
`ss-consensus/src/supervision.rs`	Réduit à la lecture d'état PostgreSQL (plus de promotion par le nœud)
`README.md`, `CLAUDE.md`, `ACCES-INTERFACES.md`, `CONTRAT-API.md`, `Architecture_SaaS_Souverain_Trois_Acteurs.md`	Réécrits une fois la refonte stabilisée

4.3 À supprimer

Élément	Raison
`spike/node/src/stock/` et les routes HTML liées au stock	Métier AMANE remplacé par le SaaS externe
`core-rust/ss-client/src/invariants.rs`	Invariants propres au stock
`frontend/src/pages/StockPage.tsx`	Écran du métier supprimé
`ss-consensus/src/quorum.rs`	Élection écrite à la main, remplacée par etcd/Patroni (ADR-001). **Retrait reporté à l'étape 2** : la supprimer avant l'arrivée d'etcd/Patroni dans la pile PME laisserait les clusters sans bascule automatique
`proto/framework.proto`	Vestige V1, même paquet que le contrat qui fait autorité
Console Django en double : `templates/**/*.html`, `views_web.py`, `views_espace.py`, `dashboard/views*.py`, `urls.py` web, `static/`	Doublon de la console React (couverture vérifiée écran par écran). Les `.tmpl` (générateurs d'installeurs) sont conservés. **Correction** : les `views.py` de `tenants/`, `licenses/` et `devices/` sont les ViewSets DRF de l'API et restent en place
`spike/node/Dockerfile.stub`, `.gitkeep` inutiles	Vestiges
`fix_locks.py`, `verify.py`, `docker-cleanup.ps1`	Scripts ponctuels ; `fix_locks.py` contient un mot de passe en clair
`.agenthub/sessions/`	Sans rapport avec le projet
`paroles_explication_mission_b.txt`	Note personnelle
`impact_missions_v1_vers_v2 (1).md`, `Stack_Technique_Framework_Souverainete_SaaS (1).md`, `schema_soutenance.html`	Déplacés vers `docs/archive/` plutôt que supprimés (le schéma de soutenance n'a pas d'équivalent React)

5. Plan de travail par étapes

Étape	Contenu	Statut
0	Branche `main3`, instantané V2, présent document	Terminée
1	Nettoyage : suppressions du § 4.3, un commit par catégorie	Terminée (sauf `quorum.rs`, reporté à l'étape 2)
2	HA de production : etcd + Patroni + orchestrateur Go dans la pile PME ; retrait de `quorum.rs` et de la promotion par le nœud ; réplication synchrone effective (ADR-001)	À faire
3	Nœud → hôte d'exécution : manifeste, lancement et supervision d'une image Docker, reverse proxy de l'interface (ADR-002)	À faire
4	Confinement et chiffrement : réseau interne sans sortie, données de l'application chiffrées au repos, sauvegardes chiffrées via le relais (ADR-003)	À faire
5	SaaS éditeur : modèle Application, soumission d'image par digest, génération des installeurs	À faire
6	Console React : page Applications, mise à jour Architecture	À faire
7	Documentation : README, CLAUDE.md, accès, contrat d'API	À faire
Plus tard	Binaires Linux et .exe Windows (ADR-002)	Prévu

6. Journal des étapes

Étape 0 — Préparation (2026-10-28)

- Création de la branche `main3` depuis `main2` (`e0f6d09`) et publication sur `origin/main3`.
- Commit `6da5022` « chore: état V2 avant nettoyage » : 76 fichiers V2 jusque-là non suivis ou non commités (pages React clusters, relais, espace tenant ; API DRF clusters, relais, statistiques, images ; quorum `ss-consensus` ; documents d'accès et de déploiement).
- Inventaire du dépôt et tri initial (§ 4).
- Décisions ADR-001, ADR-002 et ADR-003 actées.
- Création du présent document et de son générateur PDF.

Étape 1 — Nettoyage (2026-10-28)

Quatre commits, un par catégorie, pour que chacun reste lisible et réversible.

Commit	Contenu
`7ca8e78`	Racine : retrait de `fix_locks.py` (mot de passe en clair), `verify.py`, `docker-cleanup.ps1`, `.agenthub/`, d'une note personnelle ; deux notes de cadrage archivées dans `docs/archive/`
`81d7081`	Vestiges : `proto/framework.proto` (V1), `spike/node/Dockerfile.stub`, `.gitkeep` inutiles, dossier `infra/` vide. `CLAUDE.md` et le serveur gRPC pointent vers le seul proto restant
`5609fb6`	Métier stock : module `spike/node/src/stock/`, `ss-client/src/invariants.rs`, `StockPage.tsx` (routée nulle part). La racine `/` du nœud affiche une page « Application hébergée », futur point d'entrée du reverse proxy (étape 3). Authentification et console cluster du nœud conservées
`dc06197`	Console Django : vues et gabarits HTML retirés ; les fonctions partagées avec l'API passent dans des modules `services.py` (historique conservé), la voie PME par jeton dans `tenants/api_pme.py`. Aucune URL d'API ne change

Choix faits pendant l'étape.

- `quorum.rs` reste en place jusqu'à l'étape 2 : le retirer avant qu'etcd/Patroni n'arrivent dans la pile PME supprimerait toute bascule automatique chez les clients.
- Les tables `articles` et `mouvements_stock` déjà créées dans les bases PME ne sont pas supprimées : le nœud ne les crée plus, mais effacer des données client n'est pas du ressort d'un nettoyage de code.
- Le schéma de soutenance, sans équivalent React, est archivé plutôt que supprimé.

Vérifications.

- Django : `manage.py check` sans erreur ; 75 tests exécutés sur SQLite en mémoire, 74 réussis. L'échec (`test_options_de_replication_postgres`) existait déjà avant le nettoyage (vérifié sur le commit `098f1de`), voir § 7.
- Frontend : `npm run build` et `npm run lint` sans erreur.
- Go : `go build ./...` sans erreur.
- Rust : **non compilé** pendant l'étape (Docker arrêté sur le poste, aucun toolchain local) ; diff relu à la main. Compilation confirmée ensuite, voir les compléments ci-dessous.

Étape 1 — Compléments (2026-10-28)

Compilation Rust (via Docker, `rust:1.88-bookworm`).

- `spike/` : compile, 74 tests réussis, 0 échec.
- `core-rust/ss-client` : compile, 7 tests unitaires réussis.
- Les avertissements restants (`delete_session` jamais appelée, champ `node_id` jamais lu, variable `e` inutilisée dans `ss-journal`) existaient avant le nettoyage ; ceux du module stock ont disparu avec lui.

Test d'intégration `core-rust/ss-client/tests/mock_server.rs` retiré. Il implémentait le contrat V1 (`RegisterNode`, `WriteOperation`, `GetClusterStatus`) alors que `build.rs` génère les bindings depuis le contrat V2 : 25 erreurs de compilation, déjà présentes avant le nettoyage (vérifié sur `098f1de`). Il empêchait `cargo test` de lancer les tests unitaires de `ss-client`. Une version alignée sur le contrat V2 pourra être écrite si le besoin revient.

Test `test_options_de_replication_postgres` corrigé. Le test attendait `max_wal_senders=3` et `wal_keep_size=64` ; le gabarit compose (comme `pme-deploy/`) utilise 5 et 512. Le second écart était masqué par l'arrêt du test au premier échec. Les valeurs du gabarit font foi :

- `max_wal_senders=5` : deux standbys en flux et un `pg_basebackup -Xs` (qui occupe deux émetteurs) doivent pouvoir coexister sur un cluster de trois nœuds ; avec 3, le reclonage d'un standby peut être refusé.
- `wal_keep_size=512` (Mo) : un standby redémarré ou brièvement coupé rattrape son retard sans être recloné ; 64 Mo n'offrent pas cette marge.

Le test est aligné et commenté. Suite Django : 75 tests, 75 réussis.

Constat reporté à l'étape 2. En analysant ce test, la réplication s'est révélée asynchrone de fait (`synchronous_commit=remote_write` sans `synchronous_standby_names`). Correction décidée pour l'étape 2, consignée dans ADR-001.

7. Points ouverts

Sujet	Question	Lié à
Chiffrement au repos de PostgreSQL	Quel mécanisme : volume chiffré (LUKS, gocryptfs) avec clé dérivée de la DEK, ou chiffrement applicatif ? PostgreSQL n'offre pas de TDE natif	ADR-003
Licence du SaaS externe	Comment l'application vérifie-t-elle sa propre licence sans accès sortant vers son éditeur ? Relais de métadonnées par le nœud ?	ADR-003
Mises à jour de l'application	Signature des images (cosign) en plus du digest ? Qui valide une nouvelle version ?	ADR-002
Authentification utilisateur	Le reverse proxy ajoute-t-il un contrôle d'appareil appairé devant l'interface, ou laisse-t-on uniquement le login de l'application ?	ADR-003
Applications multi-conteneurs	Un SaaS peut nécessiter plusieurs services (web, worker, cache) : le manifeste doit-il décrire un ensemble ?	ADR-002
Bases autres que PostgreSQL	Que faire d'une application qui exige MySQL ou MongoDB ?	ADR-001
Réplication synchrone et disponibilité	Avec `synchronous_mode_strict`, un cluster de 2 nœuds dont le standby tombe bloque toute écriture. Faut-il l'accepter, ou n'activer le mode strict qu'à partir de 3 nœuds ? Le réglage est global au cluster (Patroni) et l'application externe n'étant pas modifiée, il ne peut pas être affiné par transaction	ADR-001, étape 2

Annexe — Mettre à jour ce document

1. Modifier la source `docs/refonte-v3/AMANE-Refonte-V3.md` : statut des étapes (§ 5), entrée dans le journal (§ 6), nouvelle ADR (§ 3) ou point ouvert (§ 7).
2. Incrémenter la version dans l'en-tête et ajouter une ligne à l'historique des versions.
3. Régénérer le PDF : `python docs/refonte-v3/generer_pdf.py` (dépendance : `reportlab`).
4. Commiter la source et le PDF ensemble, par exemple `docs(refonte): étape 1 — nettoyage`.